

Ciclo de Vida del Desarrollo de Sistemas

El SDLC define las etapas que guian el analisis, diseno, desarrollo e implementacion de sistemas de informacion de forma sistematica, reduciendo riesgos y costos.



9

FASES

PROFESOR

Carlos Godoy

ALUMNOS

Cardozo Mariano

Manavella Lautaro

Picone Ian



HERRAMIENTAS

01



Figma



React



Android



iOS

Incorporacion de la Interaccion Humano-Computadora

La HCI debe considerarse desde las primeras etapas del ciclo. El analista evalua como los usuarios interactuaran con el sistema: disenando interfaces intuitivas en Figma, respetando flujos ergonomicos, asegurando tiempos de respuesta adecuados y garantizando accesibilidad universal. Incorporar estas consideraciones desde el inicio reduce drasticamente los costos de rediseño posterior y multiplica la tasa de adopcion del sistema por parte de los usuarios finales, evitando el rechazo al cambio tecnologico.

HERRAMIENTAS

02



Jira



Confluence



Slack



Miro

Identificacion de Problemas, Oportunidades y Objetivos

El analista examina la organizacion para detectar los problemas que limitan su desempeno, las oportunidades de mejora y los objetivos estrategicos que el sistema debe cumplir. Se realizan entrevistas en Zoom o MS Teams, se centraliza el relevamiento en Confluence y se gestionan las tareas en Jira. Esta fase determina si el proyecto es viable y define el alcance real del sistema a desarrollar.



HERRAMIENTAS

03



Confluence



Notion



Miro



MS Teams

Determinacion de los Requerimientos del Factor Humano

Se relevan los requerimientos de informacion que los usuarios necesitan del sistema: que datos se requieren, en que formato, con que frecuencia y con que nivel de detalle. Se aplican entrevistas estructuradas documentadas en Confluence, cuestionarios en Notion y sesiones colaborativas en Miro. El resultado es una especificacion de requerimientos funcionales y no funcionales validada y firmada por todos los stakeholders involucrados en el proyecto.

HERRAMIENTAS

04



Figma



Miro



Jira



Confluence

Analisis de las Necesidades del Sistema

Con los requerimientos relevados, el analista modela el sistema actual y el sistema propuesto mediante diagramas de flujo de datos, diagramas entidad-relacion y casos de uso. Se usa Figma para prototipos visuales, Miro para DFDs colaborativos y Jira para rastrear cada requerimiento. El analisis produce el modelo logico del sistema, completamente independiente de la tecnologia que se usara para implementarlo.



HERRAMIENTAS

05



Figma



PostgreSQL



AWS



GitHub

Diseno del Sistema Recomendado

El analista transforma el modelo logico en un modelo fisico: define la arquitectura del sistema en capas, disenando la base de datos en PostgreSQL o MySQL, las interfaces en Figma y la infraestructura cloud en AWS. Se documenta todo en Confluence. Los cambios en esta etapa cuestan 10 veces menos que en produccion, lo que hace que invertir tiempo en un buen diseno sea la decision mas rentable del ciclo de vida del sistema.

HERRAMIENTAS

06



Python



Docker



Git



GitHub

Desarrollo y Documentacion del Software

El equipo codifica el sistema en Python, Node.js u otro lenguaje elegido en el diseno. Se usa Git y GitHub para control de versiones, Docker para entornos reproducibles y Jira para gestionar sprints y tareas. El analista supervisa que el codigo sea fiel al diseno y coordina la generacion de la documentacion tecnica (arquitectura, APIs) y la documentacion del usuario final (manuales, tutoriales y guias de referencia rapida).



HERRAMIENTAS

07



Jira



Selenium



Jest



GitHub

Prueba y Mantenimiento del Sistema

El sistema es sometido a pruebas exhaustivas: unitarias con Jest o Selenium, de integracion, de sistema completo y de aceptacion del usuario (UAT). Los errores se registran y rastrean en Jira. Se verifica el rendimiento bajo carga real y la seguridad ante vulnerabilidades. El mantenimiento posterior incluye correcciones de errores, actualizaciones de seguridad, adaptaciones al entorno y mejoras de funcionalidades solicitadas por los usuarios a lo largo del tiempo.

HERRAMIENTAS

08



Docker



Kubernetes



AWS



GitLab

Implementacion y Evaluacion del Sistema

La puesta en produccion puede ser directa, paralela, por fases o piloto. Se usa Docker y Kubernetes para orquestar el despliegue en AWS, garantizando alta disponibilidad. La evaluacion post-implementacion mide si el sistema cumple los objetivos definidos al inicio: se comparan KPIs, se recopila feedback via Slack o Teams y se identifican mejoras para el siguiente ciclo. La evaluacion cierra el SDLC e inicia el mantenimiento continuo.



El Impacto del Mantenimiento en el SDLC



AWS



Grafana



Kubernetes



Jira

El mantenimiento es la fase mas larga y costosa del SDLC: consume entre el 60% y el 80% del presupuesto total durante la vida util del sistema. Herramientas como AWS CloudWatch y Grafana permiten monitorear el sistema en tiempo real, detectando anomalias antes de que afecten a los usuarios.

El mantenimiento se divide en cuatro tipos: correctivo (reparar errores reportados via Jira), adaptativo (ajustar el sistema a cambios en el entorno tecnologico o legal), perfectivo (mejorar funcionalidades existentes) y preventivo (refactorizacion y actualizacion de dependencias para evitar fallas futuras).

El impacto es bidireccional: el mantenimiento reinicia el SDLC generando nuevos analisis y desarrollos, mientras que su costo depende directamente de la calidad del trabajo en las fases anteriores. Un sistema bien disenado y documentado puede costar hasta 100 veces menos de mantener que uno construido sin metodologia.

60-80%

del presupuesto total

4 tipos

correctivo · adaptativo
perfectivo · preventivo

x100

costo sin metodologia

Reinicia

el ciclo SDLC